

## Etapa 01: Escolha o Problema

**Problema escolhido:** Criar uma função em Python que calcule a média de notas de um aluno e retorne somente as notas que estão acima dessa média.

---

## Etapa 02: Primeira Iteração (Vaga) e Resposta Ruim

**Prompt Vago:** “Gere um código de média em Python por favor.”

**Documentação da Resposta Ruim:**

Python

# Lista de números

```
numeros = [10, 20, 30, 40, 50]
```

# Cálculo da média

```
media = sum(numeros) / len(numeros)
```

```
print(f"A média é: {media}")
```

- **Por que é ruim?** O código anterior não foi o melhor porque carecia de robustez (tratamento de erros) e modularização (uso de funções), elementos essenciais para a qualidade e reutilização em LLMs.

---

## Etapa 03 : Aplicando a Equação do Prompt

Vou estruturar agora o **Prompt Mestre** utilizando os pilares solicitados:

- **Persona:** Desenvolvedor Python Sênior
  - **Contexto:** Objetivo é gerar um código de cálculos baseado no peso das notas de cada atividade e fazê-lo dividir por 80 para obter o resultado real de sua nota.
  - **Restrições:** Utilizar Type Hints, incluir Docstrings no formato Google, tratar exceções para listas vazias ou com dados não numéricos, e não utilizar bibliotecas externas (como NumPy).
  - **Formato:** Saída em Markdown estruturado com explicação detalhada.
-

## Etapa 04: Refinamento Iterativo (Resultado Perfeito)

### Prompt Mestre Final:

“Atue como um Desenvolvedor Python Sênior. Preciso de uma função para um sistema escolar que receba uma lista de notas (floats/ints). A função deve calcular 4 notas onde a média aritmética baseada como nota peso ‘80’ e retornar um dicionário contendo a média calculada e uma sublista apenas com notas que foram estritamente superiores á média. Restrições: Utilizar Type Hints, incluir Docstrings no formato Google, trate o erro de divisão por zero caso a lista venha vazia e garanta que o código siga os princípios de Clean Code. Formate o resultado em Markdown”

### Resultado Gerado (Exemplo de Saída Esperada):

```
from typing import List, Dict, Union

def processar_notas_escolares(notas: List[Union[float, int]]) -> Dict[str,
Union[float, List[float]]]:
    """
    Calcula a média aritmética ponderada (peso 80) e filtra notas acima da média.

    Args:
        notas: Uma lista contendo números inteiros ou decimais representando as
        notas.

    Returns:
        Um dicionário com a 'media' (float) e 'notas_acima' (list) das notas
        estritamente
        superiores à média calculada.

    Raises:
        ValueError: Caso a lista de notas esteja vazia, impedindo o cálculo da média.
    """
    if not notas:
        raise ValueError("A lista de notas não pode estar vazia para o cálculo da
        média.")
```

```
# O peso 80 na média aritmética simples é aplicado como fator (0.8)

# sobre a soma ou resultado final, conforme a lógica de sistemas escolares.

media_base = sum(notas) / len(notas)

media_ponderada = round(media_base * 0.8, 2)

notas_superiores = [float(nota) for nota in notas if nota > media_ponderada]

return {

    "media": media_ponderada,

    "notas_acima": notas_superiores

}

# Exemplo de uso:

# notas_turma = [10, 7.5, 9, 6]

# print(processar_notas_escolares(notas_turma))
```

---

## Script de Explicação (O que foi feito)

Este script resume o processo realizado para a Prof<sup>a</sup>. Kadidja Vália:

### 1. Identificação da Falha

O código inicial era uma implementação genérica e frágil: carecia de tratamento para listas vazias (causando `ZeroDivisionError`), não possuía tipagem estática (Type Hints) e ignorava padrões de documentação profissional, dificultando a manutenção em sistemas de larga escala.

### 2. Engenharia do Prompt

Foi aplicada a técnica de **Persona e Contexto Profissional**. Ao definir o papel como "Desenvolvedor Python Sênior" e especificar requisitos técnicos detalhados (formato Google, restrições de tipos e estrutura de dicionário), o prompt forçou o modelo a sair de uma resposta trivial para uma entrega de nível corporativo.

### 3. Estabelecimento de Limites

Definiram-se restrições claras para o escopo da função:

- **Entrada:** Apenas ints e floats.
- **Lógica:** Cálculo específico com peso '80'.
- **Segurança:** Bloqueio de execução em listas vazias via exceção explícita.
- **Saída:** Retorno estritamente superior à média (comparação lógica >).

### 4. Entrega Estruturada

A solução foi organizada seguindo o padrão Clean Code:

- **Modularização:** Lógica encapsulada em função reutilizável.
- **Documentação:** Docstrings completas para autocompletar em IDEs.
- **Tipagem:** Uso de Union e List para garantir integridade dos dados.

### 5. Refinamento e Feedback

O processo incluiu a validação da lógica de filtragem (list comprehension) e o arredondamento da média para duas casas decimais, garantindo que o resultado final fosse tanto legível para o usuário quanto processável por outros módulos do sistema.